

Best First Forecast Source for European City Temperature Contracts

Executive summary

The best city to start with is **London** ¹, and the best first forecast source to implement is **Open-Meteo** queried at the exact contract station, **London City Airport (EGLC)**, in local time (`Europe/London`) with `daily=temperature_2m_max` in Celsius. That combination gives you the strongest balance of live market relevance, station representativeness, implementation speed, schema simplicity, and future extensibility. London is currently the strongest observed European temperature market on Polymarket, its rules explicitly resolve on the London City Airport station, and that airport is only about 5 miles from the city centre, which is unusually close for a contract-resolution airport station. Open-Meteo gives you direct daily Tmax as JSON, timezone-aware daily dates, simple Python integration, and immediate migration options to UKMO, ECMWF, GFS, and later city-specific official feeds. ²

If you insist on a strictly primary national source from day one, the best official single-city alternative is **Met Office Global Spot** for London. It offers daily point forecasts via API, but it is less convenient for a pan-European pipeline, requires registration/subscription, and is best treated as the first official source to add after the initial Open-Meteo implementation. ³

The most important design choice is this: **build around the contract's named observation station, not around a generic city centroid**. For the European cities reviewed here, current Polymarket rules resolve on airport stations such as London City (EGLC), Madrid-Barajas (LEMD), Paris-Le Bourget (LFPB), Schiphol (EHAM), Warsaw Chopin (EPWA), and Malpensa (LIMC). That means airport-vs-urban representativeness matters operationally only insofar as it affects forecast bias; settlement logic follows the named station. ⁴

What the contracts actually measure

Recent European Polymarket temperature contracts do **not** settle on “the city’s temperature” in a loose sense. They settle on the highest temperature recorded at a named airport weather station on a specified date, with Wunderground used as the resolution source and whole-degree Celsius used for settlement. London contracts resolve on London City Airport (EGLC); Madrid on Adolfo Suárez Madrid-Barajas (LEMD); Paris on Paris-Le Bourget (LFPB); Amsterdam on Schiphol (EHAM); Warsaw on Chopin (EPWA); and Milan on Malpensa (LIMC). ⁴

The label structure is also consistent enough to engineer against: these are typically **one-degree buckets with open-ended tails**. A London contract on May 12, 2026, for example, offered outcomes from “10°C or below” through single-degree buckets to “20°C or higher.” That matters because your analysis layer should think in terms of a forecast distribution over discrete Celsius bins, not just a single point estimate. ⁵

This leads to one of the most important implementation rules in the whole project: when a contract names a station, **forecast that station**. If you instead forecast “central London” or “central Madrid,” you may capture the human-climate intuition of the city but still miss the contract because the airport microclimate, exposure, and rounding rules differ. That is why airport-vs-urban station selection is a settlement problem first and a meteorology problem second. This is an inference from the published market rules and station-specific resolution design. ⁴

A further operational lesson comes from Paris. French authorities investigated suspicious temperature spikes at Paris Charles de Gaulle in April 2026, and current Paris contracts now point to Paris-Le Bourget instead. Even if your forecast model is good, you still need a versioned station registry and explicit contract-to-station mapping because contract sources can change. ⁶

Comparison of the top European city candidates

City	Country	Nearest official station(s) with IDs and distance	Polymarket event examples or likelihood	Recommended primary forecast API	Pros	Cons
London ¹	United Kingdom ⁷	London City Airport Station EGLC , about 5 miles / 8 km from city centre	“Highest temperature in London on May 12?”; London was also the top temperature market on the platform in recent listings	Open-Meteo first; Met Office next	Strongest current market presence, very close settlement station, easiest exact v1 code path, excellent official upgrade path	Estuary/marine effects can cause sharp bin-edge misses; official Met Office path is UK-only ⁸

City	Country	Nearest official station(s) with IDs and distance	Polymarket event examples or likelihood	Recommended primary forecast API	Pros	Cons
Madrid ⁹	Spain ¹⁰	Adolfo Suárez Madrid-Barajas LEMD, 12 km from city centre	"Highest temperature in Madrid on May 12?" with explicit Barajas station rules	AEMET if Spain-only; Open-Meteo for easiest integration	Active market presence, strong official Spanish open-data ecosystem, dry-regime forecasting often simpler than the UK	Airport is materially farther from the urban core; AEMET is API-keyed and more cumbersome for a pan-European first implementation ¹¹
Amsterdam ¹²	Netherlands ¹³	Schiphol EHAM, 4.9 NM SW of Amsterdam	"Highest temperature in Amsterdam on May 11?"; explicit Schiphol station rules	Open-Meteo first; KNMI later	Very good airport proximity, strong Dutch official data platform, good expansion candidate	KNMI operational model access is file/GRIB-oriented and heavier than JSON APIs ¹⁴
Warsaw ¹⁵	Poland ¹⁶	Warsaw Chopin EPWA, 8 km from city centre	"Highest temperature in Warsaw on May 11?"; explicit Chopin station rules	Open-Meteo	Good station proximity, clear station mapping, recent active contracts	Official English-language forecast API path is less obvious than for the UK, Spain, or the Netherlands ¹⁷

City	Country	Nearest official station(s) with IDs and distance	Polymarket event examples or likelihood	Recommended primary forecast API	Pros	Cons
Paris ¹⁸	France ¹⁹	Paris-Le Bourget LFPB, 7 km north of Paris	"Highest temperature in Paris on May 12?"; current rules use Le Bourget	Open-Meteo first; Météo-France later	Good station proximity and active market presence	Recent station-integrity/source-switch history makes it a worse first city operationally; official public-data portal is in migration and more complex ²⁰
Milan ²¹	Italy ²²	Malpensa LIMC, about 49 km to Milano Centrale	"Highest temperature in Milan on May 12?"; explicit Malpensa station rules	Open-Meteo	Recent market activity exists	Settlement station is too far from the urban core, so representativeness is materially worse than the other candidates; not a good v1 city ²³

Priority order: London, Madrid, Amsterdam, Warsaw, Paris, Milan. London wins because it is the best overlap of market activity, station closeness, English-language operational tooling, and low-friction implementation. Madrid is the best second city if you want to add one official national source early. Amsterdam is the best third because Schiphol is close and KNMI is excellent, but the API ergonomics are heavier. Milan falls to the bottom because Malpensa is too far from the city for a clean v1 settlement proxy.

²⁴

Forecast source assessment

The best **implementation-first** source is **Open-Meteo**. It returns direct JSON, supports `daily=temperature_2m_max`, requires `timezone` for daily values, supports Celsius, and allows forecast horizons up to 16 days. Its free non-commercial tier needs no API key but is capped at 10,000 calls per day, 5,000 per hour, and 600 per minute, with commercial options available later. It also exposes multiple upstream national and global models behind one schema, which is ideal for a later ensemble step.

²⁵

The best **official London-specific** source is **Met Office Weather DataHub Global Spot**. It provides daily point forecasts for a single latitude/longitude location via API, has a free plan up to 360 calls per day, and is the cleanest official upgrade for London once the first pipeline is live. Its drawbacks are registration/subscription friction and poor portability outside the UK. ²⁶

The best **official station-oriented** source for later expansion is **DWD MOSMIX**. MOSMIX is not just another grid model download: it is a statistically optimized point-forecast product for about 5,400 locations worldwide, with 10-day horizons and hourly updates on the short-range window. For a trading system that settles on named stations, that is conceptually excellent. The reason I still do **not** recommend it as your first implementation is engineering friction: KML/KMZ, station-list mapping, and less convenient normalization than a simple JSON API. It is, however, probably the most interesting second-wave source after Open-Meteo.

27

ECMWF Open Data and **Copernicus CDS** are excellent for high-quality official modelling and for backtesting, but they are not the best first operational city-forecast path. ECMWF Open Data is distributed in GRIB2 at 0.25° resolution, with 3-hourly and 6-hourly maximum-temperature products and a Python downloader (`ecmwf-opendata`), which means you must decode GRIB and aggregate to the contract day yourself. Copernicus CDS is outstanding for historical and reanalysis workflows, but it is not the simplest route for daily operational station-like short-range city forecasts.

28

NOAA GFS is globally available and official, but it suffers from the same first-version problem as ECMWF: you get gridded model data, not a clean daily station-like JSON forecast. It runs four times daily and is distributed through NOMADS and other official channels, but it is better treated as an upstream model inside Open-Meteo or as a later direct-source addition.

29

Among country-specific official APIs, **AEMET OpenData** is strong for Spain, **KNMI Data Platform** is strong for the Netherlands, and **Météo-France** has expanded public data access through its API portal and `meteo.data.gouv.fr`. All are valuable future additions, but all are heavier than Open-Meteo for a first deployable pipeline. AEMET is REST-based and free but requires an API key; KNMI is a file-based Open Data API with strong rate limits and CC BY 4.0 licensing; Météo-France public data is free and expanding, but the public-data estate is still being migrated and is less frictionless for a small station-Tmax pipeline.

30

Two sources should **not** be your first forecast implementation. **Meteostat** is excellent for historical/climate/station metadata work, but its developer docs describe it as historical weather and climate data rather than an operational forecast API. **meteoblue** and **DTN/MeteoGroup** both offer capable forecast APIs, but they are commercial, contract- or credit-based, and therefore worse fits for a first, narrow v1 city pipeline. DTN is especially interesting later because it explicitly works from the nearest official weather station and supports daily/hourly forecasts, but that is a second- or third-wave source, not the first.

31

Recommended source stack

Source	Coverage and Tmax handling	Access and licensing	Best use in your roadmap
Open-Meteo	Direct daily <code>temperature_2m_max</code> JSON, local timezone handling, simple station-coordinate queries	No key for low-volume non-commercial use; CC BY 4.0; commercial plans later	Implement first 32
Met Office DataHub	Official point daily forecast, good London fit	Registration; free 360 calls/day, then paid	Add second for London official confirmation 3

Source	Coverage and Tmax handling	Access and licensing	Best use in your roadmap
DWD MOSMIX	Official station-style point forecasts, 10 days, large station coverage	Open-data server, but KML/KMZ	Add after v1 for station-aligned Europe tests ³³
ECMWF Open Data / CDS	Official European/global model data; daily Tmax requires post-processing	GRIB2, <code>ecmwf-opendata</code> , <code>cdsapi</code>	Add for high-quality official model comparison and backtests ³⁴
AEMET OpenData	Spain-focused official REST/open-data ecosystem	API key required	Best official source when you add Madrid ³⁵
KNMI Data Platform	Dutch model and observation datasets via API	API key; strong rate limits; CC BY 4.0	Best official source when you add Amsterdam ³⁶
Meteostat	Historical/climate API and Python library, not operational forecast-first	RapidAPI; 500 free monthly calls	Use for backtesting and bias correction, not v1 forecast fetch ³⁷

Final recommendation and migration path

The single best first implementation is:

City: London

Station: London City Airport (`EGLC`)

Forecast source: Open-Meteo

Query pattern: station coordinates + `daily=temperature_2m_max` + `temperature_unit=celsius` + `timezone=Europe/London` ³⁸

That recommendation is not just about convenience. It fits the full product logic:

- London is the strongest currently observed European temperature market, so you get immediate live relevance. ³⁹
- The settlement station is explicit and close to the city, so forecast-target ambiguity is low. ⁴⁰
- Open-Meteo's schema is the easiest exact fit to a `ForecastValues(latitude, longitude, date, values_c)` normalization contract. ⁴¹
- You can later add official city-native sources without changing the downstream interface. ⁴²

The migration path should be:

1. **Ship London + Open-Meteo first.**
2. Add **London + Met Office** as the first official second source.

3. Add **Madrid + AEMET** and **Amsterdam + KNMI** next.
4. Add **ECMWF** or **DWD MOSMIX** as cross-city official comparison sources.
5. Then build **source ensembling and station-level bias correction** by city, lead time, and season.

43

For bias correction and ensembling, the right rule is simple: preserve a **contract-station registry** and never let the data source choose the settlement target implicitly. Your station mapping should be versioned by contract slug, with explicit fields for city label, station name, ICAO, timezone, and coordinates. Paris is the cautionary example: the relevant airport source can change. 44

Implementation checklist

The first pipeline should do only six things:

- Parse and store the contract's settlement station from the market rules.
- Fetch a **local-time, Celsius, daily Tmax** forecast for that station coordinate.
- Normalize the result into a tiny stable structure.
- Persist the raw payload for audit/debug.
- Feed the normalized values into `analysis.py`.
- Backtest against the exact resolved station later for bias correction. 45

The timeline below is the right level of scope for a first London pipeline.

```

timeline
    title London first-city pipeline
    Contract registry : Store city label, station name, ICAO=EGLC,
    timezone=Europe/London, lat/lon
    Fetch forecast : Call Open-Meteo daily Tmax at station coordinates
    Normalize : Build ForecastValues(latitude, longitude, date, values_c)
    Validate : Check timezone, Celsius, target date, plausible range, missing
    values
    Analyze : Feed normalized object into analysis.py
    Backtest later : Compare against resolved station observations and compute
    bias tables

```

The validation path should stay boring and strict.

```

flowchart TD
    A[Fetch raw forecast payload] --> B{HTTP 200 and JSON?}
    B -- No --> X[Retry or fail source]
    B -- Yes --> C{Timezone matches contract day?}
    C -- No --> X1[Reject payload]
    C -- Yes --> D{Temperature unit is Celsius?}
    D -- No --> X2[Convert or reject]
    D -- Yes --> E{Target date present in daily series?}
    E -- No --> X3[Reject payload]

```

```
E -- Yes --> F{Value plausible for weather?}
F -- No --> X4[Flag anomaly]
F -- Yes --> G[Emit ForecastValues to analysis.py]
```

Minimal Python example

For the recommended first source, you can implement the whole fetch-and-normalize step with a single small function. London City Airport coordinates are approximately **51.5053, 0.0553**. 46

```
from dataclasses import dataclass
from typing import List
import requests

@dataclass(frozen=True)
class ForecastValues:
    latitude: float
    longitude: float
    date: str
    values_c: List[float]

def fetch_open_meteo_daily_tmax_for_date(
    latitude: float,
    longitude: float,
    target_date: str,
    timezone: str,
) -> ForecastValues:
    """
    Fetch daily Tmax from Open-Meteo and normalize to ForecastValues.

    target_date must be ISO format: YYYY-MM-DD
    """
    url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "daily": "temperature_2m_max",
        "temperature_unit": "celsius",
        "timezone": timezone,
        "forecast_days": 7,
        # Keep defaults at first. Test cell_selection="nearest" later during
        # backtests.
        "cell_selection": "land",
    }
```



```

response = requests.get(url, params=params, timeout=20)
response.raise_for_status()
payload = response.json()

daily = payload["daily"]
series = dict(zip(daily["time"], daily["temperature_2m_max"]))

if target_date not in series:
    raise KeyError(f"Target date {target_date} not found in forecast")

value_c = float(series[target_date])

# Basic sanity check for European city Tmax contracts
if not (-40.0 <= value_c <= 60.0):
    raise ValueError(f"Implausible Tmax value: {value_c}")

return ForecastValues(
    latitude=float(payload["latitude"]),
    longitude=float(payload["longitude"]),
    date=target_date,
    values_c=[value_c], # later: append more sources/models here
)

# Example: London City Airport (EGLC) for a target contract date
forecast = fetch_open_meteo_daily_tmax_for_date(
    latitude=51.5053,
    longitude=0.0553,
    target_date="2026-05-12",
    timezone="Europe/London",
)

print(forecast)

```

This matches Open-Meteo's documented daily Tmax parameterization, local-time daily handling, and Celsius output. ⁴⁷

A minimal handoff into `analysis.py` can remain equally small:

```

def build_analysis_payload(city: str, station_id: str, forecast:
ForecastValues) -> dict:
    return {
        "city": city,
        "station_id": station_id,
        "date": forecast.date,
        "values_c": forecast.values_c,
    }

```

```

        "latitude": forecast.latitude,
        "longitude": forecast.longitude,
    }

analysis_payload = build_analysis_payload(
    city="London",
    station_id="EGLC",
    forecast=forecast,
)

```

The right way to grow this later is to keep `ForecastValues` stable and add more `values_c` entries from additional sources or models, rather than changing your downstream interface. That makes ensembling and calibration much easier. This is an implementation recommendation based on the API and market rule structure above. ⁴⁸

Open questions and limitations

Some official national APIs were assessed at a product and documentation level, but their complete response schemas were not fully validated here; that is why the exact code example is provided only for Open-Meteo. Distances are not fully harmonized geodesic measurements from one single method; in a few cases they come from operator-published airport-to-city references or rail/airport references such as Milano Centrale. Finally, the city ranking is based on high-confidence live/recent market evidence from May 11, 2026, not a fully paginated historical Gamma archive.

¹ ²⁸ ³⁴ <https://www.ecmwf.int/en/forecasts/datasets/open-data>
<https://www.ecmwf.int/en/forecasts/datasets/open-data>

² ²⁴ ³⁹ Temperature Predictions & Real-Time Odds
https://polymarket.com/predictions/temperature?utm_source=chatgpt.com

³ ¹² ²⁶ ⁴² ⁴³ Weather DataHub | Home
<https://datahub.metoffice.gov.uk/>

⁴ ⁵ ⁷ ⁸ ¹⁰ ¹⁸ ¹⁹ ²² ³⁸ ⁴⁰ ⁴⁵ ⁴⁸ Highest temperature in London on May 12? Trading Odds & Predictions (May 12, 2026) | Polymarket
<https://polymarket.com/event/highest-temperature-in-london-on-may-12-2026>

⁶ ⁹ ⁴⁴ <https://www.theguardian.com/world/2026/apr/23/hairstyler-or-lighter-french-police-look-at-claim-of-sensor-tampering-to-win-weather-bets>
<https://www.theguardian.com/world/2026/apr/23/hairstyler-or-lighter-french-police-look-at-claim-of-sensor-tampering-to-win-weather-bets>

¹¹ Highest temperature in Madrid on May 12? Trading Odds & Predictions (May 12, 2026) | Polymarket
<https://polymarket.com/event/highest-temperature-in-madrid-on-may-12-2026>

¹³ ²³ Highest temperature in Milan on May 12? Trading Odds & Predictions (May 12, 2026) | Polymarket
<https://polymarket.com/event/highest-temperature-in-milan-on-may-12-2026>

14 Highest temperature in Amsterdam on May 11? Trading Odds & Predictions (May 11, 2026) | Polymarket
<https://polymarket.com/event/highest-temperature-in-amsterdam-on-may-11-2026>

15 25 32 41 47 <https://open-meteo.com/en/docs>
<https://open-meteo.com/en/docs>

16 31 37 <https://dev.meteostat.net/>
<https://dev.meteostat.net/>

17 Highest temperature in Warsaw on May 11? Trading Odds & Predictions (May 11, 2026) | Polymarket
<https://polymarket.com/event/highest-temperature-in-warsaw-on-may-11-2026>

20 Highest temperature in Paris on May 12? Trading Odds & Predictions (May 12, 2026) | Polymarket
<https://polymarket.com/event/highest-temperature-in-paris-on-may-12-2026>

21 46 <https://www.weather.gov/wrh/timeseries?site=eglc>
<https://www.weather.gov/wrh/timeseries?site=eglc>

27 33 https://www.dwd.de/EN/ourservices/met_application_mosmix/met_application_mosmix.html
https://www.dwd.de/EN/ourservices/met_application_mosmix/met_application_mosmix.html

29 <https://www.ncei.noaa.gov/products/weather-climate-models/global-forecast>
<https://www.ncei.noaa.gov/products/weather-climate-models/global-forecast>

30 <https://opendata.aemet.es/>
<https://opendata.aemet.es/>

35 https://www.aemet.es/en/datos_abiertos
https://www.aemet.es/en/datos_abiertos

36 <https://developer.dataplatform.knmi.nl/open-data-api>
<https://developer.dataplatform.knmi.nl/open-data-api>